Jarvis Part 4 by Pulse of AI

Official Step-by-Step Guide & Reference Code

Important Prerequisite Notice

Stop! This tutorial is **Part 4** of the series. If you have not completed **Part 3** (setting up your project environment, installing base dependencies like google-genai and pyaudio, and setting up your Google AI Studio API key), **this script will not work**. Make sure your environment is fully configured from Part 3 before proceeding.

Reference Links

- **Part 3 Code & Setup:** Google Drive Code File
- **Series Video Reference:** YouTube Video Short
- **Official Creator Channel:** Pulse of AI YouTube Channel

Implementation Guide

Step 1: Verify your .env file contains your API key.
Step 2: Paste the reference code below into your agent.py file inside VS Code.
Step 3: Run your assistant using: python agent.py

Reference Code (agent.py)

```
import os
import sys
import asyncio
import pyaudio
from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()
client = genai.Client(api_key=os.getenv("GOOGLE_API_KEY"))

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512
p = pyaudio.PyAudio()

model_is_speaking = False

async def send_audio(session):
    global model_is_speaking
    mic_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_IN, input=True, frames_per_buffer=CHUNK)
    print("■ [LISTENING] Speak into your mic...")
    try:
        while True:
            data = await asyncio.to_thread(mic_stream.read, CHUNK, exception_on_overflow=False)
            if model_is_speaking:
                continue
            await session.send_realtime_input(audio={"data": data, "mime_type": "audio/pcm;rate=16000"})
    except asyncio.CancelledError:
        raise
```

```
    finally:
        try: mic_stream.stop_stream()
        except OSError: pass
        try: mic_stream.close()
        except OSError: pass

async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty(): audio_queue.get_nowait()
                        model_is_speaking = False
                    if server_content.interim_input_transcription:
                        print("\r[HEARD] " + server_content.interim_input_transcription.text, end="",
flush=True)
                    if server_content.input_transcription:
                        print("\r[FINAL] " + server_content.input_transcription.text)
                    model_turn = server_content.model_turn
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            print("■ [SPEAKING] Jarvis is answering...")
                        for part in model_turn.parts:
                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)
                    if server_content.turn_complete:
                        model_is_speaking = False
                        print("■ [LISTENING] Ready for your next question...")
    except asyncio.CancelledError:
        raise
    finally:
        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(format=FORMAT, channels=CHANNELS, rate=RATE_OUT, output=True,
frames_per_buffer=CHUNK)
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None: break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:
        try: speaker_stream.stop_stream()
        except OSError: pass
        try: speaker_stream.close()
        except OSError: pass

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    config = types.LiveConnectConfig(
```

```
        response_modalities=["AUDIO"],
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(prefix_padding_ms=100,
silence_duration_ms=300)
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(prebuilt_voice_config=types.PrebuiltVoiceConfig(voice_name="Puck")
)
        ),
        system_instruction=types.Content(
            parts=[types.Part.from_text(
                text="You are JARVIS, an advanced artificial intelligence assistant created by Swayam
Dhawale, Founder of Pulse of AI. You are currently in your active development stage. Respond in extremely
short, crisp sentences. Never use markdown or filler words. If asked about your identity or creator, state
clearly that you were built by Swayam Dhawale."
            )]
        )
    )
    print("All protocols activated, Jarvis initialized...")
    async with client.aio.live.connect(model=model_id, config=config) as session:
        print("Made by Swayam Dhawale, Founder of Pulse of AI.")
        audio_queue = asyncio.Queue()
        send_task = asyncio.create_task(send_audio(session))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))
        try:
            await asyncio.gather(send_task, receive_task, playback_task)
        finally:
            send_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(send_task, receive_task, playback_task, return_exceptions=True)

if __name__ == "__main__":
    if sys.platform == "win32":
        asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy())
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("\nDisconnected.")
```